

National Gallery Route Finder

Project Summary

This project is a JavaFX application for finding routes around the main floor (level 2) of the National Gallery, London. The system uses graph data structures and traversal algorithms to calculate different route types between rooms or exhibits.

The project models:

- Rooms as graph nodes
- Connections between rooms as weighted edges
- Exhibits as artworks stored inside rooms

The application supports graph-based routing and pixel-based breadth-first search on a black-and-white traversal map.

Graph Model

``Room``

Each room stores:

- ``id``
- ``name``
- ``category``
- ``interestScore``
- ``featuredArtists``
- ``exhibits``
- ``connections``

The `connections` list is the adjacency list for the room.

``Edge``

Each edge stores:

- destination room
- distance
- interest score

This allows the project to use weighted route calculations.

`Graph`

The graph stores all rooms and provides methods to:

- add rooms
- create directed connections
- create undirected connections

`Exhibit`

Exhibits are stored inside rooms and allow the user to choose either a room or an exhibit as a route endpoint.

Database Files

The project uses editable CSV files:

- `rooms.csv`
- `connections.csv`
- `exhibits.csv`
- `room\_coordinates.csv`

These files are manually editable, which satisfies the assignment requirement for a suitable updateable database representation.

Algorithms

Single Valid Route

The service layer can return a single route between two selected points/rooms/exhibits by using the graph algorithms and resolving exhibit selections back to their containing rooms.

DFS

Depth-first search is used to generate multiple valid route permutations between two locations.

BFS

There are two BFS-related uses in the project:

- graph BFS for shortest route by number of room-to-room steps

- pixel BFS for shortest route across the map image

The pixel BFS works on a generated black-and-white traversal image where white pixels are treated as accessible and black pixels are treated as blocked.

Dijkstra

Dijkstra's algorithm is used for:

- shortest route by weighted distance
- most interesting route using interest weighting and preferred artists

Route Features

The system supports:

- room or exhibit start and end selections
- multiple routes with DFS
- shortest route with BFS
- shortest route with Dijkstra
- most interesting route with Dijkstra
- waypoint support
- avoiding specified rooms/exhibits

JavaFX GUI

The GUI includes:

- start and destination selectors
- preferred artist input
- waypoint input
- avoid-room input
- max-routes input
- route output area
- graph overview area
- map display
- route overlay drawing
- black-and-white map toggle
- pixel BFS controls

Testing and Benchmarking

JUnit

The project includes JUnit tests for:

- graph algorithms
- route service behavior
- sample graph loading
- pixel BFS

JMH

The project includes JMH benchmarks for:

- DFS
- BFS
- Dijkstra shortest route
- Dijkstra interesting route
- service-level route methods

Marking Scheme Coverage

- Custom graph data structure/classes: covered
- Single valid route: covered
- Multiple valid route permutations using DFS: covered
- Shortest route using Dijkstra: covered
- Shortest route using BFS and illustrating route on map: covered
- Most interesting route using Dijkstra: covered
- Waypoint support: covered
- Avoiding specified rooms/exhibits: covered
- JavaFX GUI: covered
- JUnit testing: covered
- JMH benchmarking of key methods: covered
- General completeness, structure, commenting, and logic: covered

Verification

The Maven test suite passed successfully:

```
mvn -q test
```

Short Demo Explanation

The project represents the gallery as a graph. Each room is a node and each connection between rooms is an edge. DFS is used to find multiple possible routes, BFS is used for shortest route searches, and Dijkstra is used for weighted shortest and most interesting routes. A separate pixel BFS is used on the map image to find a shortest traversable visual path and draw it back onto the map.